

Spring Boot JWT Authentication

Detailed learning guide - exact flows and code concepts discussed

Register -> Login -> AuthenticationManager -> ProviderManager -> DaoAuthenticationProvider -> Authentication -> JWT -> JWT verification -> Authorization -> Refresh Token -> Logout

This guide is written as a study document for the code and questions we worked through. It emphasizes the actual object flow, which class calls which class, what is inside Authentication, how providers are selected, what a JWT contains, and how refresh-token based sessions work.

Topic	What you will understand
Registration	How a user is created, password hashed, roles linked, and permissions reached
Login	How UsernamePasswordAuthenticationToken reaches DaoAuthenticationProvider
Authentication internals	ProviderManager scanning providers and interface/reference dispatch
JWT	Header, payload, claims, signature, expiration, encoding vs signing
Protected APIs	JwtAuthenticationFilter, SecurityContextHolder, authenticated vs authorized
OTP	Custom AuthenticationToken + AuthenticationProvider + OtpService
Refresh tokens	Short-lived access JWT + sliding refresh + rotation + absolute session
Logout / frontend	401 vs 403, refresh-on-401, logout when refresh fails

1. Big picture

The system has three major request stages.

- Register: create the user in the database and store the password as a BCrypt hash. Roles/permissions are linked to the user.
- Login: username/password is authenticated through AuthenticationManager and DaoAuthenticationProvider. On success, an access JWT and (in the refresh-token design) a refresh token are returned.
- Protected API: the client sends the access JWT. JwtAuthenticationFilter validates it, loads UserDetails, creates Authentication, stores it in SecurityContextHolder, and then authorization rules decide whether the request can continue.

REGISTER

Client -> RegisterController -> AuthService -> PasswordEncoder -> UserRepository -> DB

LOGIN

Client -> SecurityFilterChain -> JwtFilter (no JWT, continue)
-> AuthController -> AuthService
-> AuthenticationManager -> ProviderManager
-> DaoAuthenticationProvider -> UserDetailsService -> DB
-> PasswordEncoder -> Authentication SUCCESS
-> JwtService -> Access JWT (+ RefreshTokenService -> RefreshToken)

PROTECTED API

Client + Bearer JWT -> SecurityFilterChain -> JwtAuthenticationFilter
-> JwtService -> UserDetailsService -> Authentication
-> SecurityContextHolder -> Authorization -> Controller

A critical distinction: authentication answers "Who are you?" while authorization answers "Are you allowed to do this?"

2. Register flow

Registration is not the same as login. Registration creates the account. Login proves the credentials and issues an access token.

Typical register request

```
POST /auth/register
Content-Type: application/json
```

```
{
  "username": "sumanth",
  "password": "password123"
}
```

File-by-file flow

- RegisterController: receives the HTTP request and creates a RegisterRequest DTO.
- AuthService: validates the request and creates a User entity.
- PasswordEncoder: converts the raw password into a BCrypt hash. The raw password should not be stored.
- UserRepository: saves the User entity.
- Role relationship: the user is linked to a role. The role links to permissions through role_permissions.

Core register code

```
User user = new User();
user.setUsername(request.getUsername());

String hashedPassword =
    passwordEncoder.encode(request.getPassword());

user.setPassword(hashedPassword);
user.getRoles().add(userRole);

userRepository.save(user);
```

Database picture

```
users
-----
id | username | password_hash
1 | sumanth | $2a$10$...

user_roles
-----
user_id | role_id
1       | 2

role_permissions
-----
role_id | permission_id
2       | 1
2       | 5
```

The important chain is: User -> Role -> Permission. During authentication, those permissions can become Spring Security authorities.

3. Login flow - exact execution

Login code

```
public LoginResponse login(LoginRequest loginRequest) {  
    Authentication authentication =  
        authenticationManager.authenticate(  
            new UsernamePasswordAuthenticationToken(  
                loginRequest.getUserName(),  
                loginRequest.getPassword()  
            )  
        );  
  
    return new LoginResponse(  
        jwtService.generateToken(  
            authentication.getName()  
        )  
    );  
}
```

Step 1 - LoginRequest

```
loginRequest.getUserName() -> "sumanth"  
loginRequest.getPassword() -> "password123"
```

Spring/Jackson created LoginRequest from the JSON body. At this point this is only data from the client.

Step 2 - create UsernamePasswordAuthenticationToken

```
new UsernamePasswordAuthenticationToken(  
    "sumanth",  
    "password123"  
)
```

This is an authentication request object. Conceptually it contains principal/username = sumanth, credentials = password123, no authorities yet, and authenticated = false.

Step 3 - pass token to AuthenticationManager

The declared type is AuthenticationManager, an interface. The runtime object is typically Spring Security ProviderManager. Java interface polymorphism means the call is dispatched to the actual ProviderManager object.

```
AuthenticationManager authenticationManager;  
  
// runtime conceptually:  
authenticationManager ---> ProviderManager
```

4. How AuthenticationManager reaches DaoAuthenticationProvider

This was one of the most important internal questions. The key is that ProviderManager holds a list of AuthenticationProvider references.

```
ProviderManager
  providers
    |
    +--> DaoAuthenticationProvider
    +--> AnotherAuthenticationProvider
    +--> ...
```

Conceptual internal logic

```
for (AuthenticationProvider provider : providers) {
    if (provider.supports(authentication.getClass())) {
        Authentication result =
            provider.authenticate(authentication);

        if (result != null) {
            return result;
        }
    }
}
```

This is a conceptual simplification of the real Spring Security implementation, but it captures the provider-selection mechanism: check supports(), then call authenticate() on a provider that can handle the token.

Exact dry run

```
AuthService
  |
  | authenticationManager.authenticate(TOKEN_A)
  v
ProviderManager.authenticate(TOKEN_A)
  |
  v
provider[0] = DaoAuthenticationProvider
  |
  | supports(UsernamePasswordAuthenticationToken.class)
  v
true
  |
  v
DaoAuthenticationProvider.authenticate(TOKEN_A)
  |
  +--> UserDetailsService
  |
  +--> PasswordEncoder
  |
  v
authenticated Authentication returned
  |
  v
ProviderManager -> AuthService
```

Java reference idea

Think of the relationship as object references, not a magical jump. AuthService variable -> ProviderManager object -> providers list -> DaoAuthenticationProvider object -> authenticate(token).

5. What is actually inside Authentication?

Before authenticate()

UsernamePasswordAuthenticationToken TOKEN_A

```
-----  
principal      = "sumanth"  
credentials    = "password123"  
authorities    = empty  
authenticated  = false
```

This means: "Here are my credentials. Please authenticate me."

After successful authenticate()

Authentication TOKEN_B

```
-----  
principal      = CustomUserDetails  
credentials    = null / cleared  
authorities    = ROLE_ADMIN, USER_READ, ...  
authenticated  = true
```

This means: "The credentials have been verified. This is the authenticated user and these are the authorities."

Important methods

Method	Meaning
authentication.getName()	Gets the authenticated name, normally from the principal/UserDetails
authentication.getPrincipal()	The authenticated identity object, such as CustomUserDetails
authentication.getAuthorities()	The granted roles/permissions
authentication.isAuthenticated()	Whether the Authentication represents successful authentication

After successful authentication, authentication.getName() is typically "sumanth" and that value is fed into JwtService.generateToken().

6. DaoAuthenticationProvider - real work

What it does

- Receives UsernamePasswordAuthenticationToken.
- Calls UserDetailsService.loadUserByUsername(username).
- Gets UserDetails, including password hash and authorities.
- Uses PasswordEncoder/BCryptPasswordEncoder to compare the entered password with the stored hash.
- If valid, returns an authenticated Authentication object.

```
DaoAuthenticationProvider
|
+--> CustomUserDetailsService
|   |
|   +--> UserRepository
|   |   |
|   |   +--> Database
|   |
|   +--> PasswordEncoder
|       |
|       +--> BCryptPasswordEncoder
```

CustomUserDetailsService

```
User user = userRepository.findByUsername(username)
    .orElseThrow(() ->
        new UsernameNotFoundException(
            username + " not found"
        ));
```

```
return new CustomUserDetails(user);
```

Repository returns Optional. orElseThrow() extracts the User if present or throws if absent.

Password check

```
passwordEncoder.matches(
    "password123",
    "$2a$10$..."
)
```

```
// true -> authentication succeeds
// false -> authentication fails
```

7. SecurityFilterChain configuration

```
@Bean
public SecurityFilterChain securityFilterChain(
    HttpSecurity http,
    AuthenticationProvider authenticationProvider)
    throws Exception {

    http
        .csrf(csrf -> csrf.disable())

        .sessionManagement(session ->
            session.sessionCreationPolicy(
                SessionCreationPolicy.STATELESS
            )
        )

        .authenticationProvider(
            authenticationProvider
        )

        .authorizeHttpRequests(auth -> auth
            .requestMatchers(
                "/auth/register",
                "/auth/login"
            ).permitAll()

            .anyRequest().authenticated()
        )

        .addFilterBefore(
            jwtAuthenticationFilter,
            UsernamePasswordAuthenticationFilter.class
        );

    return http.build();
}
```

Why each part exists

Configuration	Meaning
csrf.disable()	For this stateless learning REST API, CSRF is disabled. Browser/cookie applications require more
STATELESS	Do not use the normal HTTP session to persist authentication between requests.
authenticationProvider(...)	Registers/configures the AuthenticationProvider that participates in the username/password authentication
permitAll()	Login/register can be called before the user has an access token.
anyRequest().authenticated()	Every other request needs an authenticated SecurityContext.
addFilterBefore(...)	Places JwtAuthenticationFilter before UsernamePasswordAuthenticationFilter in the security chain

You normally create/configure the provider as a Spring bean. You do not normally create the ProviderManager/provider list yourself; Spring Security manages that infrastructure.

8. JWT - components and generation

JWT stands for JSON Web Token. In our design it is the short-lived access credential. A JWT has three dot-separated parts: HEADER.PAYLOAD.SIGNATURE.

```
eyJhbGciOiJIUzI1NiJ9
.
eyJzdWIiOiJzdW1hbnRoIiwiaWF0IjoiLi4uIiwiaXhwIjoiLi4uIn0
.
SIGNATURE
```

Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

alg describes the signing algorithm. typ identifies the token type.

Payload

```
{
  "sub": "sumanth",
  "iat": "...",
  "exp": "...",
}
```

sub = subject, iat = issued-at time, exp = expiration time. The payload is readable because JWT encoding is not encryption.

Signature

For an HMAC-signed JWT, the server uses the header + payload and a secret key to calculate a signature. JWT handles the cryptographic details when `.signWith(secretKey)` is called.

```
Jwts.builder()
  .subject(username)
  .issuedAt(issuedAt)
  .expiration(expiration)
  .signWith(secretKey)
  .compact();
```

Encoding is not encryption. Anyone who has the token can decode the header/payload. Never put passwords, OTPs, database passwords, or other secrets inside the payload.

9. JwtAuthenticationFilter - future requests

The JWT filter is mainly for requests made after login. It looks for the Bearer token and, if valid, creates Authentication and puts it into SecurityContextHolder.

```
String authHeader =
    request.getHeader("Authorization");

if (authHeader == null ||
    !authHeader.startsWith("Bearer ")) {

    filterChain.doFilter(request, response);
    return;
}

String token = authHeader.substring(7);

String username;
try {
    username = jwtService.extractUsername(token);
} catch (Exception e) {
    filterChain.doFilter(request, response);
    return;
}

if (username != null &&
    SecurityContextHolder.getContext()
        .getAuthentication() == null) {

    UserDetails userDetails =
        userDetailsService.loadUserByUsername(username);

    boolean valid = jwtService.isTokenValid(
        token,
        userDetails.getUsername()
    );

    if (valid) {
        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(
                userDetails,
                null,
                userDetails.getAuthorities()
            );

        SecurityContextHolder.getContext()
            .setAuthentication(authentication);
    }

    filterChain.doFilter(request, response);
}
```

Why filterChain.doFilter()?

A filter must either continue the chain or deliberately terminate the request. In our implementation, no JWT means "I did not authenticate this request using JWT; let the rest of Spring Security decide what to do."

No JWT vs invalid JWT vs valid JWT

Situation	Filter result	Later result
No Authorization header	Continue without Authentication	Protected endpoint normally -> 401
Malformed/invalid JWT	Continue without Authentication	Protected endpoint normally -> 401
Valid JWT	Create Authentication + SecurityContext	Authorization can run

10. Protected API flow and 401 vs 403

Example: GET /users

GET /users
Authorization: Bearer <access-token>

- JwtAuthenticationFilter validates the access JWT.
- UserDetailsService loads the user and authorities.
- Authentication is stored in SecurityContextHolder.
- .anyRequest().authenticated() checks that the request is authenticated.
- @PreAuthorize("hasAuthority('USER_READ')") checks the required authority.
- If allowed, the controller executes.

Roles vs authorities

```
@PreAuthorize("hasRole('ADMIN')")
```

```
// checks for ROLE_ADMIN
```

If your CustomUserDetails converts the database role ADMIN to the authority ROLE_ADMIN, then hasRole("ADMIN") succeeds. Role/authority expression names are case-sensitive: use hasRole, not HasRole.

Status codes

Status	Meaning	Example
401 Unauthorized	Authentication failed/missing	No JWT, expired JWT, invalid JWT, refresh failed
403 Forbidden	Authenticated but not authorized	Student has COURSE_READ but endpoint needs COURSE_CR

Typical frontend decision

```
API request
|
+--> 200 -> success
|
+--> 401 -> try /auth/refresh once
|
|           +--> success -> replace tokens, retry original request
|           +--> failure -> clear auth state and go to login
|
+--> 403 -> do NOT refresh; user lacks permission
```

11. Custom AuthenticationProvider - OTP example

A custom AuthenticationProvider is useful when the credential mechanism is different from the normal username/password mechanism. OTP is one example.

Password login

```
UsernamePasswordAuthenticationToken
    |
    v
AuthenticationManager -> ProviderManager
    |
    v
DaoAuthenticationProvider
    |
    +--> UserDetailsService
    +--> PasswordEncoder
```

OTP login

```
OtpAuthenticationToken
    |
    v
AuthenticationManager -> ProviderManager
    |
    v
OtpAuthenticationProvider
    |
    +--> OtpService
    +--> UserDetailsService
```

OtpAuthenticationToken

```
public class OtpAuthenticationToken
    extends AbstractAuthenticationToken {

    private final String phoneNumber;
    private final String otp;
    private final Object principal;

    // request token
    public OtpAuthenticationToken(
        String phoneNumber,
        String otp) {
        super(null);
        this.phoneNumber = phoneNumber;
        this.otp = otp;
        this.principal = phoneNumber;
        setAuthenticated(false);
    }

    // authenticated result
    public OtpAuthenticationToken(
        Object principal,
        Collection<? extends GrantedAuthority> authorities) {
        super(authorities);
        this.principal = principal;
        this.phoneNumber = null;
        this.otp = null;
        setAuthenticated(true);
    }
}
```

OtpAuthenticationProvider

```
@Component
public class OtpAuthenticationProvider
    implements AuthenticationProvider {

    @Override
    public Authentication authenticate(
        Authentication authentication) {

        OtpAuthenticationToken otpToken =
            (OtpAuthenticationToken) authentication;

        boolean valid = otpService.verifyOtp(
```

```

        otpToken.getPhoneNumber(),
        otpToken.getOtp()
    );

    if (!valid) {
        throw new BadCredentialsException(
            "Invalid OTP"
        );
    }

    UserDetails user =
        userDetailsService.loadUserByUsername(
            otpToken.getPhoneNumber()
        );

    return new OtpAuthenticationToken(
        user,
        user.getAuthorities()
    );
}

@Override
public boolean supports(Class<?> authenticationType) {
    return OtpAuthenticationToken.class
        .isAssignableFrom(authenticationType);
}
}

```

The custom provider does not have to generate/send the OTP itself. OtpService owns OTP-specific work such as generation, storage, expiration, and verification. The provider is the bridge into Spring Security authentication.

12. OtpService - what is inside?

For learning, the OTP can be stored in memory. A production system should normally use a secure store such as Redis with an expiry/TTL, plus rate limits and attempt limits.

```
@Service
public class OtpService {

    private final Map<String, String> otpStorage =
        new HashMap<>();

    private final SecureRandom random =
        new SecureRandom();

    public String generateOtp(String phoneNumber) {

        int otpNumber =
            100000 + random.nextInt(900000);

        String otp =
            String.valueOf(otpNumber);

        otpStorage.put(phoneNumber, otp);

        return otp;
    }

    public boolean verifyOtp(
        String phoneNumber,
        String enteredOtp) {

        String storedOtp =
            otpStorage.get(phoneNumber);

        if (storedOtp == null) {
            return false;
        }

        if (!storedOtp.equals(enteredOtp)) {
            return false;
        }

        otpStorage.remove(phoneNumber);

        return true;
    }
}
```

The OTP is removed after successful verification so it is single-use. In real production code, use a cryptographically secure generator and an expiring store; never return the OTP in the HTTP response as done in a learning demo.

Why a different token for OTP?

UsernamePasswordAuthenticationToken represents username/password. OtpAuthenticationToken explicitly represents phone/OTP, so ProviderManager can select the correct AuthenticationProvider through supports(). Reusing UsernamePasswordAuthenticationToken for OTP would mix two different credential meanings.

13. Refresh token architecture

The access JWT is intentionally short-lived. The refresh token is the longer-lived credential used only to obtain a new access token.

Login response

```
{
  "accessToken": "eyJ...",
  "refreshToken": "abc123..."
}
```

In the design we discussed, the access token is a signed JWT. The refresh token is an opaque random value stored server-side in a refresh_tokens table.

RefreshToken entity

RefreshToken

```
-----
id
token
user_id
expires_at
session_expires_at
revoked
```

user_id links the refresh token to the user. expires_at is the sliding/idle expiry. session_expires_at is the absolute maximum lifetime. revoked lets logout or rotation invalidate the token.

Our chosen example policy

Credential	Lifetime/use
Access token	15 minutes; sent with normal APIs
Refresh token idle expiry	7 days; extended on successful refresh
Absolute session lifetime	30 days; cannot be extended beyond this
Rotation	Every refresh revokes the old refresh token and creates a new one

14. Refresh flow - exact dry run

A. Login creates both tokens

Authentication SUCCESS

```
|  
+--> JwtService.generateToken(username)  
|      -> short-lived access JWT  
|  
+--> RefreshTokenService.createRefreshToken(user)  
|      -> random refresh token  
|      -> database row  
|      -> expiresAt = now + 7 days  
|      -> sessionExpiresAt = login + 30 days  
|      -> revoked = false
```

B. Normal API calls use only the access token

GET /api/courses

Authorization: Bearer <access-token>

The normal `JwtAuthenticationFilter` does not validate the refresh token on every API call. That is intentional.

C. Access token expires

Frontend -> GET /api/courses + expired access JWT

Backend -> 401 Unauthorized

Frontend -> POST /auth/refresh + refresh token

D. Backend validates refresh token

- Find refresh token by token value in `RefreshTokenRepository`.
- If not found -> fail.
- If `revoked == true` -> fail.
- If `expiresAt` is in the past -> fail.
- If `sessionExpiresAt` is in the past -> fail.
- If valid -> continue to rotation.

15. Refresh-token rotation, active users, and logout

Rotation

```
Refresh Token A
  |
  | POST /auth/refresh
  v
validate A
  |
  +--> revoke A
  |
  +--> create Refresh Token B
  |
  +--> create new Access Token
  v
Response:
{
  "accessToken": "NEW_ACCESS",
  "refreshToken": "B"
}

A = revoked
B = valid
```

Sliding expiry

If the user refreshes before the idle expiry, the newly created refresh token gets a new `expiresAt` = now + 7 days, but never beyond `sessionExpiresAt`. This lets an active user remain signed in without forcing a login every seven days.

```
Login Aug 25
  expiresAt      = Sep 1
  sessionExpiresAt = Sep 24
```

```
Refresh Aug 29
  new expiresAt  = Sep 5
  session max    = Sep 24
```

```
Refresh Sep 20
  normal new expiry = Sep 27
  BUT max session   = Sep 24
  => new expiresAt  = Sep 24
```

Logout

```
POST /auth/logout
{
  "refreshToken": "CURRENT_REFRESH_TOKEN"
}
```

```
Backend:
RefreshTokenService.revokeRefreshToken(token)
  -> revoked = true
```

Important: revoking a refresh token does not automatically make an already-issued access JWT invalid. The access JWT may remain usable until its short expiration unless you implement access-token revocation/blacklisting or another server-side check.

16. Frontend behavior - how refresh is normally called

The frontend usually does not wait for a literal message saying "JWT expired". It sees the HTTP status code 401 Unauthorized from a protected API and uses an HTTP interceptor/wrapper to handle it.

```
make API request
|
+--> 200 -> return response
|
+--> 401 -> attempt refresh once
|
|   +--> /auth/refresh -> 200
|   |
|   |   +--> save new access + refresh tokens
|   |   +--> retry original API request
|   |
|   +--> /auth/refresh -> 401
|   |
|   |   +--> clear auth state
|   |   +--> redirect to login
|
+--> 403 -> user is authenticated but lacks permission
      -> do NOT call refresh just for RBAC
```

The refresh attempt should be limited so you do not create a refresh loop. A 403 is normally an authorization failure, not an expired-token signal.

17. Why logout cannot automatically kill an access JWT

Normal access-token verification is stateless: the server verifies the JWT signature and expiration and reconstructs Authentication. It does not normally query the refresh_tokens table for every API request.

Logout at 10:30

Refresh token -> revoked

Access JWT -> still valid until, for example, 10:45

GET /api/courses with old access JWT at 10:31

-> signature valid

-> not expired

-> may be accepted

If immediate access-token revocation is required, add a server-side revocation/blacklist mechanism (often using a JWT ID/jti and a fast store such as Redis). That introduces state into the access-token validation path and is a deliberate trade-off.

Why not just make access JWT last 30 days?

A stolen access JWT would then remain usable for a much longer period. Short-lived access tokens reduce the window of exposure; refresh tokens provide session continuity.

18. Master end-to-end flow

REGISTRATION

```
-----
RegisterRequest
-> AuthController
-> AuthService
-> PasswordEncoder
-> UserRepository
-> User saved with hashed password
-> role linked -> permissions reachable
```

LOGIN

```
-----
LoginRequest
-> SecurityFilterChain
-> JwtAuthenticationFilter (no JWT; continue)
-> /auth/login permitted
-> AuthController
-> AuthService
-> UsernamePasswordAuthenticationToken (request, authenticated=false)
-> AuthenticationManager
-> ProviderManager
-> provider.supports(token.class)
-> DaoAuthenticationProvider
-> UserDetailsService
-> UserRepository / DB
-> CustomUserDetails (authorities)
-> PasswordEncoder
-> authenticated Authentication (authenticated=true)
-> JwtService -> access JWT
-> RefreshTokenService -> refresh token (refresh-token design)
```

PROTECTED API

```
-----
Request + Bearer access JWT
-> JwtAuthenticationFilter
-> extract username
-> verify JWT signature / claims / expiration
-> UserDetailsService
-> create authenticated Authentication
-> SecurityContextHolder
-> .authenticated()
-> @PreAuthorize / RBAC
-> Controller
```

REFRESH

```
-----
Expired access token -> API returns 401
-> Frontend calls /auth/refresh with refresh token
-> RefreshTokenRepository -> validate revoked/expiry/session limit
-> rotate old refresh token
-> create new access JWT
-> create new refresh token
-> frontend stores new pair
-> retry original API
```

LOGOUT

```
-----
POST /auth/logout
-> revoke current refresh token
-> future refresh denied
-> existing access JWT may live until its short expiry unless access-token revocation is implemented
```

Final mental model

Access token: "Let me use the APIs for a short time."

Refresh token: "Let me ask for another access token without making the user type the password again."

AuthenticationManager: "Route these credentials to a provider that knows how to authenticate them."

DaoAuthenticationProvider: "I authenticate username/password with UserDetailsService + PasswordEncoder."

JwtAuthenticationFilter: "For this request, inspect the Bearer JWT and, if valid, build Authentication."

SecurityContextHolder: "Store who this request is authenticated as."

Authorization: "Now check whether this authenticated user has the required role/authority."